

ESPECIALIZACIÓN EN **INTELIGENCIA DE DATOS** para la Gestión Estratégica

BASES DE DATOS, DBMS Y CLAVES

Clase 1 · Taller T8002 — Representación de datos

Docente: Dr. Lautaro Di Bartolo

Asignatura: T8002 — Representación de datos

Cohorte o comisión: Cohorte 2026

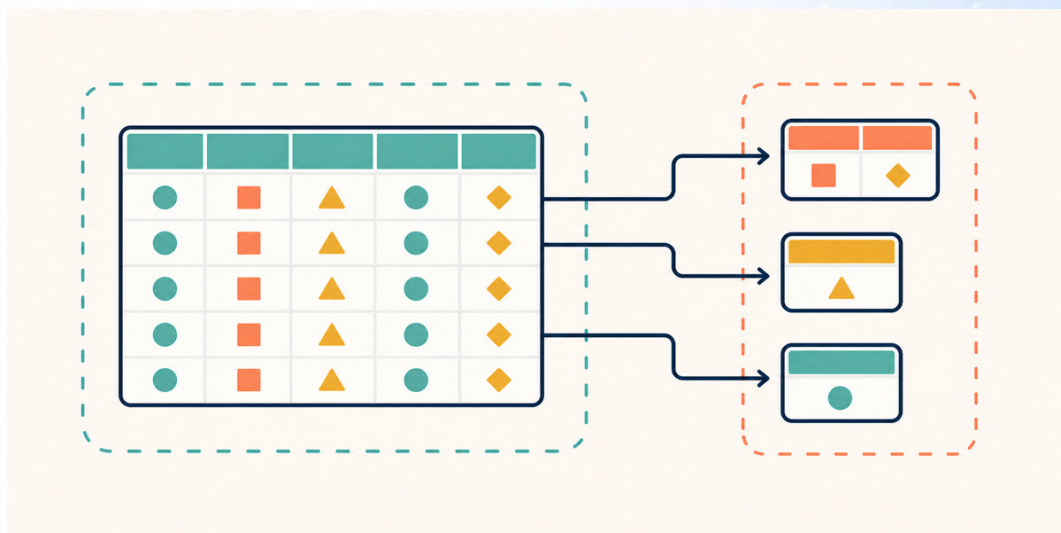


Figura 1. El mismo dato, guardado dos veces: una planilla que repite y un conjunto de tablas que se refieren.

1. INTRODUCCIÓN

1.1 Dónde quedó el T8001, y qué pregunta abre este taller

El taller anterior terminó con un archivo limpio. Las localidades escritas de siete maneras quedaron en tres, las fechas quedaron todas en el mismo formato, el duplicado exacto se fue y los valores centinela dejaron de contarse como ingresos. El resultado fueron **37 hogares** en una sola tabla, prolija, lista para analizar.

Este taller empieza mirando ese archivo prolijo y haciendo una pregunta incómoda: ¿está bien guardado?

La respuesta es que no. Un archivo puede estar impecable en sus valores y ser al mismo tiempo una mala forma de conservar el dato. Los problemas del T8001 eran de **contenido**: valores mal escritos, faltantes, repetidos. Los problemas de este taller son de **estructura**: el mismo hecho anotado en veinte lugares, un dato que no se puede cargar porque no hay dónde ponerlo, y un borrado que se lleva puesta información que nadie quería perder.

Esos problemas no se arreglan limpiando. Se arreglan cambiando la forma en que el dato está organizado. Eso es lo que este taller llama **representación de datos**, y el instrumento que la sostiene se llama base de datos.

1.2 Lo que se supone, y qué leer si no hiciste el T8001

Este apunte supone cuatro cosas del taller anterior, y ninguna es programación:

- Qué es una **tabla** de datos: filas, columnas, y un valor por celda.
- Que un archivo .csv es texto plano, y que una herramienta lo interpreta al abrirlo.

- Qué es un **diccionario de variables**: el documento que dice qué significa cada columna.
- Qué es una **escala de medición**: nominal, ordinal, de intervalo y de razón.

Los dos talleres de nivelación se pueden cursar por separado. Si no hiciste el T8001, alcanza con leer dos secciones tuyas: la 3.5 de su clase 1, que fija las tres condiciones de una tabla usable, y la 6.5 de su clase 2, que explica el diccionario de variables. La escala de medición está en la 6.2 de esa misma clase.

También aparece una palabra de pandas, merge, y una sola vez, en la sección 9. Es la operación que junta dos tablas, y la clase 2 la explica.

1.3 La tabla de este taller

Las clases 1, 2 y 3 trabajan sobre un mismo archivo, relevamiento_plano.csv. Es el relevamiento de hogares del T8001, ya limpio, con dos agregados.

El primero: **columnas que el T8001 no tenía**. La provincia de cada localidad, y el legajo, el nombre y el teléfono del encuestador que hizo la visita. Son datos que cualquier relevamiento real guarda, y que en el T8001 no hacían falta.

El segundo: **ocho hogares recibieron una segunda visita** en junio. Eso convierte el archivo en un registro de visitas y no de hogares, y es el detalle del que depende media clase.

Los números del taller, y conviene retenerlos:

Cuántos	Qué
45	filas del archivo, una por visita
12	columnas
37	hogares distintos
3	localidades: Ramallo, Córdoba y Concepción
3	provincias: Buenos Aires, Córdoba y Tucumán
4	encuestadores
4	servicios: agua, luz, gas y cloacas

Estas son las primeras cinco filas, recortadas a las columnas que importan ahora:

id_hogar	localidad	provincia	fecha_visita	encuestador_legajo	encuestador_nombre	personas
1	Ramallo	Buenos Aires	2025-03-14	E04	Sosa, Diego	3
1	Ramallo	Buenos Aires	2025-06-12	E04	Sosa, Diego	4
2	Córdoba	Córdoba	2025-03-15	E02	Ledesma, Julio	5
3	Concepción	Tucumán	2025-03-15	E03	Bianchi, Ana	2
4	Ramallo	Buenos Aires	2025-03-16	E01	Rivas, Marta	4

Mirá el hogar 1. Aparece dos veces, y en marzo tenía tres personas y en junio cuatro. No es un error: es un hogar que creció. Y mirá las dos primeras columnas: Ramallo y Buenos Aires van a aparecer juntas veinte veces en este archivo.

1.4 El alcance de este taller

Una aclaración que conviene hacer temprano, porque baja la ansiedad: **en este taller no se escribe SQL y no se implementa ninguna base de datos.**

SQL es el lenguaje con el que se le habla a una base de datos relacional. Existe, se va a nombrar y en la clase 2 vas a ver seis renglones escritos en él, para que la forma te resulte conocida cuando te la cruces. Escribirlo, crear una base, cargarla y consultarla es el contenido de la asignatura **Modelado de datos**, que viene después. Este taller es la introducción a los conceptos: qué es una base de datos, qué hace, cómo se decide su estructura y por qué esa decisión importa.

El eje es el criterio de diseño. Un mismo conjunto de datos admite varias estructuras correctas, y lo que se evalúa es la justificación de la que se eligió.

2. HERRAMIENTAS

2.1 Google Colab, otra vez

La herramienta es la misma del T8001. Colab es un servicio gratuito de Google que ejecuta código Python desde el navegador, sin instalar nada, con una cuenta de Google.

Las cuatro cosas que conviene recordar son las mismas:

- **Guardá una copia primero.** Archivo → Guardar una copia en Drive.
- **Se ejecuta en orden, de arriba hacia abajo.** Cada celda usa lo que dejaron las anteriores.
- **Cuando algo se desordena, empezá de nuevo.** Entorno de ejecución → Reiniciar y ejecutar todo.
- **Los archivos están en el panel de la izquierda,** en el ícono de carpeta.

2.2 Los notebooks de este taller solo se ejecutan

Acá hay una diferencia con el T8001, y es importante.

En el T8001 el notebook proponía cambios: había celdas marcadas para que probaras una variante y vieras qué pasaba. En este taller **no hay ninguna**. Los notebooks son demostraciones: cada celda muestra sobre datos concretos un concepto del apunte, y el resultado se lee justo abajo. No hay nada que completar ni que corregir.

La razón es el alcance. Lo que se aprende acá se aprende leyendo y mirando, no escribiendo código. El único momento en que vas a producir algo es el trabajo final, y ahí no hace falta programar.

La única biblioteca externa es **pandas**, la misma del T8001, más la biblioteca estándar de Python. Se usa para mostrar tablas, no para diseñar bases de datos. Cuando el notebook necesita mostrar qué hace una base de datos, lo imita con pandas: es una demostración, no una implementación.

3. UNA TABLA SOLA NO ALCANZA

3.1 El mismo hecho, escrito veinte veces

Volvé a la tabla de la sección 1.3 y contá cuántas veces dice que Ramallo está en la provincia de Buenos Aires.

En el archivo completo, veinte. Hay veinte visitas a hogares de Ramallo, y cada una de esas veinte filas repite el par Ramallo — Buenos Aires. Lo mismo con Córdoba, doce veces, y con Concepción, trece.

Eso se llama **redundancia**: el mismo hecho guardado más de una vez.

La redundancia no es un error de tipeo. Ninguna de esas veinte filas está mal. El problema es que el archivo guarda **un solo hecho** —Ramallo pertenece a Buenos Aires— en veinte lugares distintos, y nada garantiza que los veinte digan lo mismo.

Pasa igual con el encuestador. El legajo E04 aparece en diecisiete filas, y en cada una vuelven a escribirse su nombre y su teléfono. Diecisiete copias de un dato que es uno.

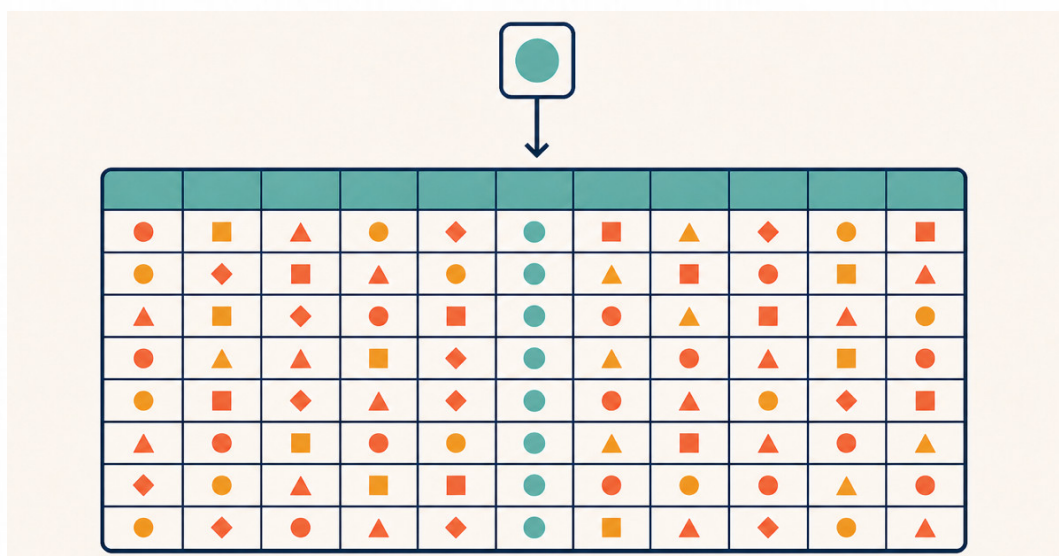


Figura 2. Un hecho, muchas copias. La redundancia no está en el valor: está en la cantidad de lugares donde vive.

3.2 La anomalía de modificación

Acá empieza a doler. El encuestador E04 cambia de teléfono.

En el archivo hay diecisiete filas con su teléfono. Para dejarlo actualizado hay que cambiar las diecisiete. Si alguien cambia dieciséis, el archivo queda diciendo dos teléfonos distintos para la misma persona, y no hay forma de saber cuál es el bueno: los dos parecen igual de legítimos.

Eso es la **anomalía de modificación**: un solo cambio en el mundo obliga a muchos cambios en el archivo, y cualquier cambio incompleto deja el archivo contradiciéndose.

Y no hace falta mala intención. Alcanza con un filtro mal puesto, un Buscar y reemplazar que no llegó al final, o dos personas editando la misma planilla en momentos distintos.

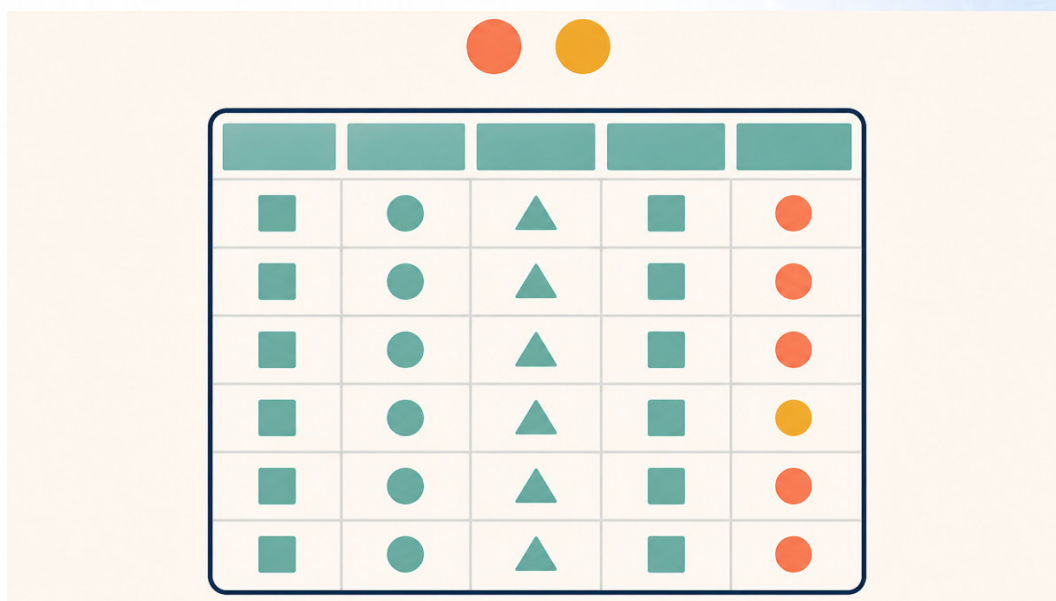


Figura 3. El cambio parcial no da ningún error. Deja dos verdades donde había una.

3.3 Las otras dos: insertar y borrar

La redundancia trae otras dos consecuencias, y las dos son por el lado opuesto.

La anomalía de inserción: no hay dónde anotar lo que todavía no pasó. El municipio decide incorporar Pergamino al relevamiento. Pergamino está en Buenos Aires, y eso ya se sabe. Pero todavía no se visitó ningún hogar, así que no hay ninguna fila donde escribirlo. Para registrar que Pergamino existe hay que inventar una fila de visita con casi todo vacío, y esa fila falsa después va a aparecer en cada conteo.

En este archivo, una localidad solo puede existir si tiene al menos una visita. El archivo no puede representar una localidad sin hogares, y eso es una limitación de la estructura y no del dato.

La anomalía de borrado: se pierde un hecho que nadie quería perder. Supongamos que las trece visitas a Concepción se anulan, porque el operativo se hizo mal y hay que rehacerlo. Se borran las trece filas, que es lo correcto. Y con ellas desaparece del archivo el único registro de que Concepción está en Tucumán, y de que Ana Bianchi es la encuestadora con el legajo E03 y ese teléfono.

Se quiso borrar una cosa y se borraron tres. El archivo no tiene forma de distinguirlas, porque las guarda en el mismo lugar.

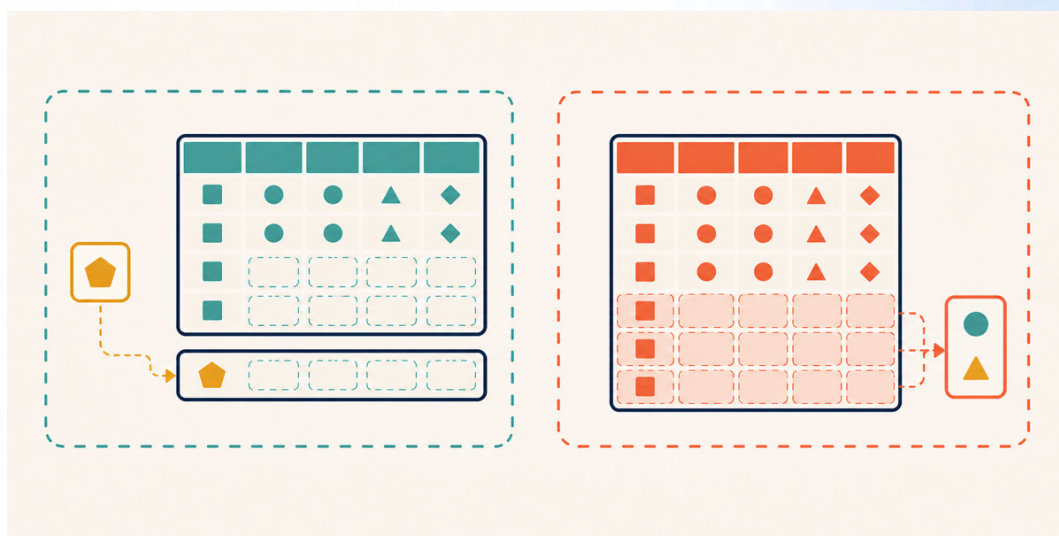


Figura 4. Las dos anomalías opuestas: lo que no entra porque falta contexto, y lo que sale de más porque compartía lugar.

Las tres anomalías tienen la misma causa y la misma cura. La causa es que hechos de naturaleza distinta comparten una fila. La cura es separarlos.

3.4 A dónde va este taller

Antes de seguir con problemas, conviene ver la salida. Este es el destino, para la parte de la tabla que venimos mirando:

Tabla localidad

id	nombre	provincia
1	Ramallo	Buenos Aires
2	Córdoba	Córdoba
3	Concepción	Tucumán

Tabla hogar

id	localidad_id
1	1
2	2
3	3
4	1

Tres filas en un lado, y en el otro un número que apunta a ellas. Ahora el par Ramallo — Buenos Aires está escrito **una vez**. Cambiarlo es cambiar una celda. Agregar Pergamino es agregar una fila, sin inventar ninguna visita. Y borrar todas las visitas a Concepción no borra Concepción.

Las tres anomalías desaparecieron, y no porque se limpió nada: porque el dato quedó guardado en el lugar que le corresponde.

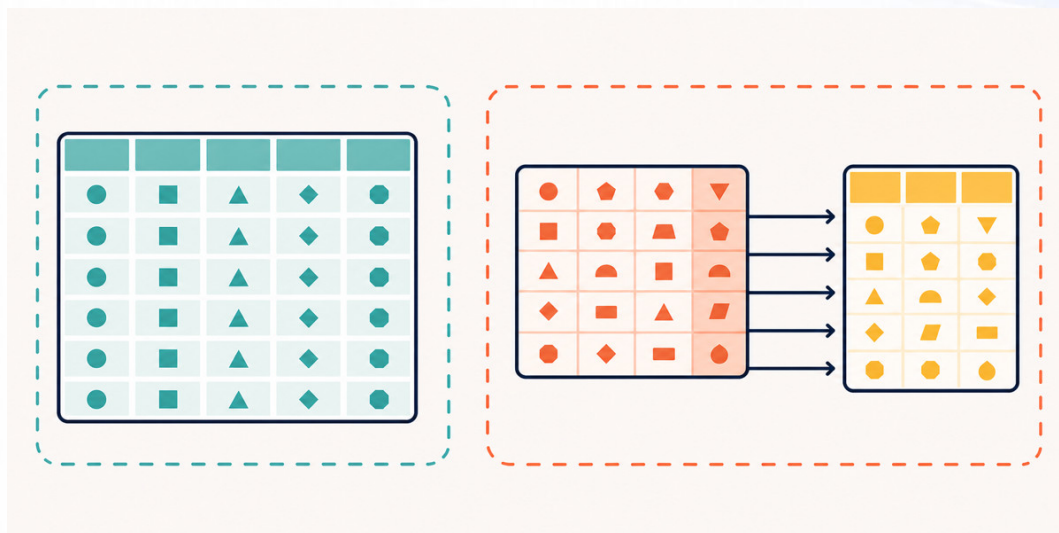


Figura 5. El arco del taller entero: de una tabla que repite a varias tablas que se refieren.

Todo el resto de este apunte es el vocabulario para describir ese dibujo, y las clases 2 y 3 son el método para llegar a él desde cualquier tabla plana.

3.5 Lo que un archivo no puede prohibir

Hay un cuarto problema, y es de otra clase. El archivo no puede impedir nada.

Nada en un .csv evita que alguien escriba Ramallo en la columna provincia, 0 -3 en personas, o una fecha de 1870 en fecha_visita. Tampoco evita que aparezcan dos filas que dicen ser el mismo hogar con datos distintos. El T8001 se topó con eso: dos filas reclamaban ser el hogar 41, con contenido diferente, y no había manera de decidir cuál era la buena. Las dos se descartaron, y el taller perdió un hogar.

Lo que falló ahí no fue la limpieza. Fue que **nadie pudo declarar la regla a tiempo**. Si el lugar donde se guardan los hogares tuviera anotado "no puede haber dos hogares con el mismo id", la segunda fila nunca habría entrado. El conflicto se habría resuelto en el momento de la carga, con la persona que tenía el papel en la mano, y no tres meses después con un analista adivinando.

Un archivo guarda valores. No guarda reglas. Y ahí está la diferencia que define este taller.

4. QUÉ ES UNA BASE DE DATOS

4.1 Qué tiene una base de datos que un conjunto de archivos no tiene

Una **base de datos** es un conjunto de datos organizado de modo que se puedan consultar y mantener sin depender de quién los escribió. Tres cosas la separan de una carpeta con archivos:

1. **Guarda las reglas junto con los datos.** No solo dice que este hogar tiene tres personas: dice que personas es un número entero positivo, que id no se puede repetir y que localidad_id tiene que apuntar a una localidad que exista. Las reglas están declaradas y se cumplen solas.
2. **Guarda los vínculos entre los datos.** Las tablas no son archivos separados que alguien junta a mano: la base sabe que hogar.localidad_id apunta a localidad.id, y lo hace respetar.
3. **La administra un programa, y no vos.** Ese programa recibe los pedidos, controla las reglas, ordena los accesos simultáneos y decide cómo se guarda todo en el disco. Es el DBMS, y es el tema de la sección 7.

De ahí sale la definición corta que conviene retener: una base de datos es **datos más reglas más un programa que las hace cumplir**.

4.2 Base de datos, archivos y planilla de cálculo

La comparación, fila por fila. La columna del medio es una planilla de cálculo, que es el caso que más se parece al trabajo real de este público:

	Archivos sueltos	Planilla de cálculo	Base de datos
Dónde vive el dato	en cada archivo	en la hoja	en tablas con estructura declarada
Dónde viven las reglas	en ningún lado	en validaciones opcionales	declaradas y obligatorias
Quién impide un valor imposible	nadie	nadie, salvo que se configure	el DBMS, siempre
Vínculo entre dos conjuntos	la arma quien lee	fórmulas de búsqueda	declarada y controlada
Dos personas escribiendo	la última gana	la última gana, o se bloquea	el DBMS ordena los turnos
Quién puede ver qué	el permiso del archivo	el permiso del archivo	por usuario, tabla y columna
Si se corta a la mitad	queda un archivo roto	queda una hoja a medias	la operación se deshace entera
Consultar sin abrir todo	no	no	sí, y es lo normal

La planilla de cálculo está más cerca de los archivos que de la base de datos, y esa es la conclusión útil. Una planilla es una herramienta excelente para mirar, calcular y presentar. Como lugar donde vive el dato de una organización tiene los problemas de la sección 3, todos.

Y ojo con la trampa de tamaño: el problema de una planilla no es que sea chica. Una planilla de cien mil filas tiene exactamente las mismas anomalías que una de cuarenta y cinco. El problema es estructural, no de volumen.

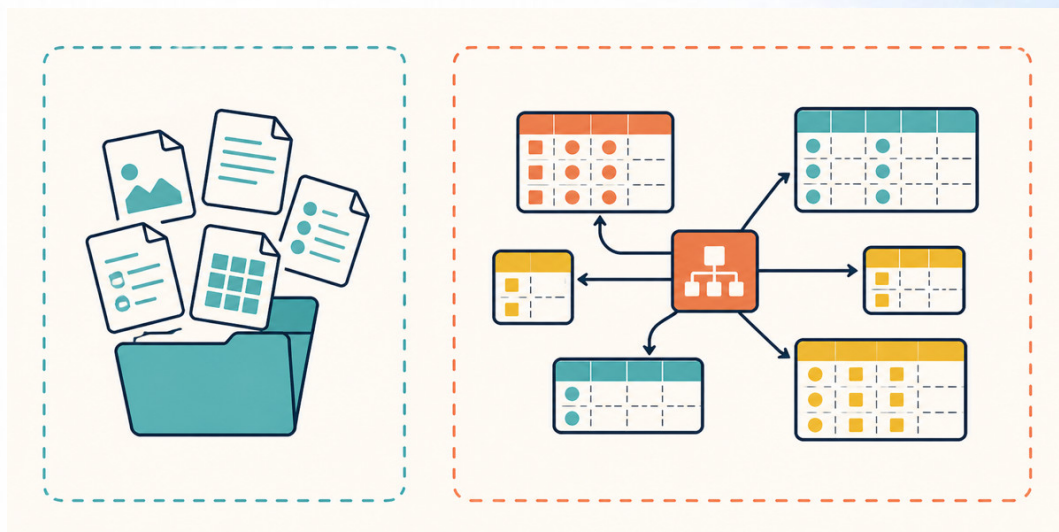


Figura 6. Del lado izquierdo cualquiera escribe cualquier cosa. Del derecho hay un programa que revisa antes de dejar entrar.

5. EL VOCABULARIO DEL MODELO RELACIONAL

Todo lo que vimos hasta acá se puede decir con las palabras de siempre: tabla, fila, columna. La teoría usa otras tres, y conviene reconocerlas porque están en todos los textos del área.

5.1 Cómo llama la teoría a la tabla, la fila y la columna

El **modelo relacional** es la forma de organizar datos que propuso Edgar Codd en 1970, y es la que usan la enorme mayoría de las bases de datos en producción. Tiene su propio vocabulario:

Palabra de siempre	Palabra de la teoría	Qué es
tabla	relación	un conjunto de filas con las mismas columnas
fila	tupla	un conjunto de valores, uno por columna
columna	atributo	una propiedad, con su nombre y su conjunto de valores posibles

Con eso, la tabla *localidad* de la sección 3.4 es una relación de tres tuplas y tres atributos.

Hay un detalle de la teoría que vale la pena, porque explica una restricción que vas a ver todo el taller: en el modelo relacional, una relación es un **conjunto**. Y en un conjunto no hay elementos repetidos ni hay orden. De ahí salen dos consecuencias que en una planilla no existen:

- **No hay dos filas idénticas.** La duplicación que el T8001 tuvo que buscar y borrar, acá está prohibida por definición.
- **Las filas no tienen número de orden.** No existe "la fila 12". Una fila se identifica por su contenido, y para eso están las claves de la sección 6.

En el cuerpo de este taller vamos a escribir **tabla**, **fila** y **columna**, porque se leen mejor. **Tupla** y **relación** aparecen solo para que las reconozcas. **Atributo** sí se usa: es la palabra que las clases 2 y 3 necesitan para definir las formas normales.

5.2 Los sinónimos, y una palabra que significa dos cosas

El T8001 dejó tres grupos de sinónimos. Este taller extiende dos y agrega el cuarto:

*variable = columna = campo = **atributo** observación = fila = caso = registro = **tupla** conjunto de datos = dataset tabla = **relación***

Y ahora la advertencia más importante de esta sección. **La palabra "relación" significa dos cosas distintas**, y las dos se usan en este taller:

- En el modelo relacional, una **relación es una tabla**. Es el sentido de esta sección.
- En el diseño conceptual de la clase 2, una **relación es un vínculo entre dos cosas**: un hogar *pertenece a* una localidad. Ahí "relación" no es una tabla: es la línea que une dos cajas.

Es la confusión más común del tema, y no se resuelve eligiendo un sentido: los dos son estándar y los vas a encontrar en cualquier texto. Se resuelve mirando el contexto. Cuando este apunte diga tabla, va a escribir **tabla**. Cuando hable del vínculo, la clase 2 lo va a avisar de nuevo.

Algo parecido pasa con **atributo**: acá es una columna de una tabla, y en la clase 2 va a ser una propiedad de una entidad. Son dos miradas de lo mismo, y la clase 2 lo aclara donde corresponde.

5.3 Dominio: el conjunto de valores admitidos

El **dominio** de una columna es el conjunto de valores que esa columna puede tomar. No qué valores tiene: qué valores **le está permitido** tener.

Los dominios del relevamiento, escritos:

Columna	Dominio
id_hogar	enteros positivos
localidad	exactamente uno de: Ramallo, Córdoba, Concepción
fecha_visita	una fecha entre marzo de 2025 y hoy
personas	enteros de 1 a 20
ingreso	número mayor o igual a 0, o sin dato
cobertura_salud	si, no, o sin dato
satisfaccion	mala, regular, buena, o sin dato

Ninguno de esos dominios está escrito en el .csv. Los sabemos porque el T8001 los averiguó y los anotó en su diccionario de variables. El archivo no los conoce, y por eso no puede hacerlos cumplir: personas = -3 entra sin una queja. Una base de datos los tiene declarados y los revisa en cada carga. Eso es la sección 5.2 de la clase 2.

El dominio no es la escala de medición. Son dos preguntas distintas sobre la misma columna, y se confunden seguido. La escala del T8001 dice qué operaciones tienen sentido: satisfaccion es ordinal, así que se puede ordenar y no se puede promediar. El dominio dice qué valores se admiten: mala, regular o buena, y nada más. Una columna puede tener la escala bien declarada y el dominio abierto, y ahí entra cualquier cosa. satisfaccion sigue siendo ordinal si alguien escribe excelente, pero su dominio quedó violado.

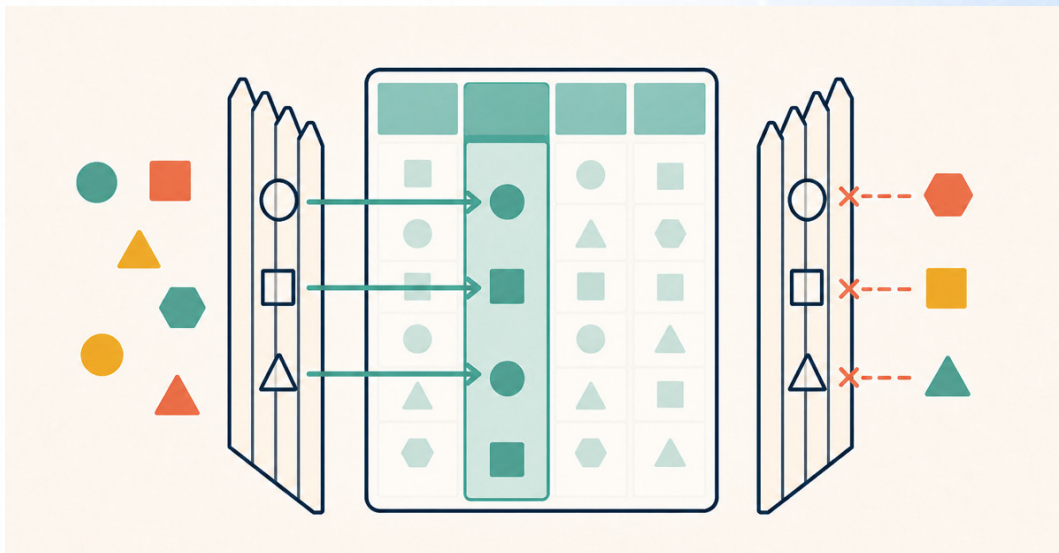


Figura 7. El dominio es el cerco. Sin cerco declarado, cualquier valor es un valor válido.

5.4 El nulo no es un valor

En el relevamiento hay celdas vacías. El ingreso de los hogares 3 y 9, y cobertura_salud y satisfaccion en diez filas. Esa ausencia se llama **nulo**.

Un nulo no es cero, no es una cadena vacía y no es un espacio. Es la marca de que **no hay valor**. Y de eso salen tres consecuencias que sorprenden:

- **Un nulo no es igual a otro nulo.** Dos hogares sin ingreso conocido no tienen "el mismo ingreso": no se sabe el de ninguno de los dos. Comparar dos desconocidos no da verdadero: da desconocido.
- **Un nulo contamina las cuentas.** Sumar un número con un dato ausente no da el número: da ausente. Las herramientas suelen saltar los nulos al sumar, y eso es una decisión de la herramienta, no de la aritmética.
- **Una clave primaria no admite nulos**, nunca. Se ve en la sección 6.2, y la razón es esta: si la columna que identifica una fila está vacía, esa fila no se puede identificar.

El T8001 llegó más lejos que esto, y vale recordarlo porque tiene una salida de diseño. Ese taller separó tres ausencias que no son la misma: **no aplica** (la pregunta no correspondía), **no contestó** (correspondía y la persona no quiso) y **no se preguntó** (correspondía y falló el instrumento). Al declararlas todas como nulos, las tres se vuelven iguales y la distinción se pierde.

Los diez nulos de cobertura_salud y satisfaccion son exactamente ese caso: caen siempre juntos, en las mismas diez filas, y el T8001 no pudo decidir entre cuatro explicaciones del instrumento. Los dejó como estaban.

Un diseño de varias tablas tiene una respuesta para eso, y no es rellenar ni borrar. La clase 2 la da cuando llega a la participación opcional, en su sección 3.5.

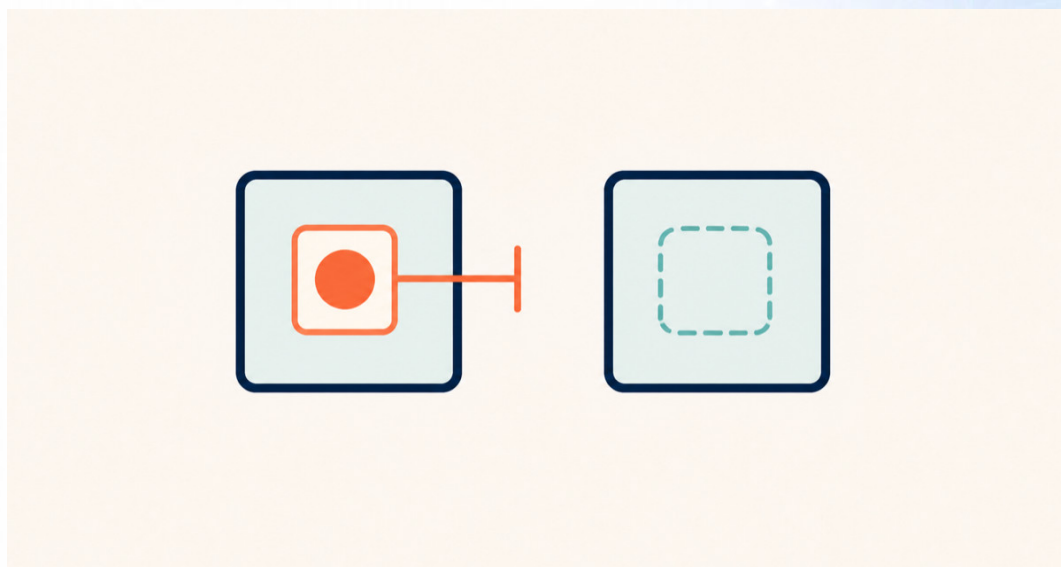


Figura 8. Un cero es una medición. Un nulo es la falta de una medición. La pantalla no los distingue.

6. CLAVES

6.1 Para qué sirve una clave

En el modelo relacional no existe "la fila 12": las filas no tienen número de orden. Entonces hace falta otra forma de señalar una fila, y esa forma es una **clave**: una o más columnas cuyo valor identifica una sola fila de la tabla.

Una clave hace tres trabajos, y los tres importan:

1. **Identificar.** Permite decir "el hogar 7" sin ambigüedad.
2. **Impedir el duplicado.** Si *id* es clave, la base rechaza una segunda fila con el mismo *id*. Es la regla que le faltó al hogar 41 del T8001.
3. **Permitir la referencia.** Otra tabla puede apuntar a esta fila guardando su clave. Sin clave no hay a dónde apuntar, y sin referencia no hay varias tablas.

6.2 Cómo se elige la clave primaria

Una **clave candidata** es cualquier conjunto mínimo de columnas que identifica una fila. Una tabla puede tener varias.

En una tabla de encuestadores hay dos: el legajo, y el documento. Las dos identifican a una persona sola. Se elige una para el uso diario, y esa es la **clave primaria**. La otra queda como **clave alternativa**: la base también le exige que no se repita, pero no es la que usan las otras tablas para referirse.

Cómo se elige entre las candidatas. Tres criterios, en este orden:

- **Que no cambie nunca.** Si la clave cambia, todas las referencias a ella quedan apuntando al vacío. Un legajo no cambia; un apellido sí.
- **Que sea corta y simple.** Se va a repetir en cada tabla que apunte a esta.
- **Que nunca esté vacía.** Es una obligación, no una preferencia: una clave primaria no admite nulos.

Y acá aparece la decisión más frecuente del diseño. Una **clave natural** es un dato del mundo que ya identifica la fila: el legajo del encuestador, el CUIT de una empresa. Una **clave sustituta** es un número que inventa la base, sin ningún significado, solo para identificar.

El relevamiento tiene el ejemplo perfecto de por qué la natural falla. Parecería que el nombre de la localidad alcanza como clave: son tres, y no se repiten. El problema es que un nombre de localidad **no es único en el país**, y este mismo archivo lo tiene: hay una Concepción en Tucumán y otra en Corrientes, y hay más de un San Martín. Hoy las tres localidades del relevamiento no se repiten y la clave funciona; el día que el operativo llegue a la segunda Concepción, la clave se rompe y hay que rehacer todas las referencias.

Por eso la tabla localidad de la sección 3.4 tiene una columna id con los valores 1, 2 y 3. Ese número no significa nada, y ahí está su virtud: **no puede quedar desactualizado, porque no describe nada del mundo.**

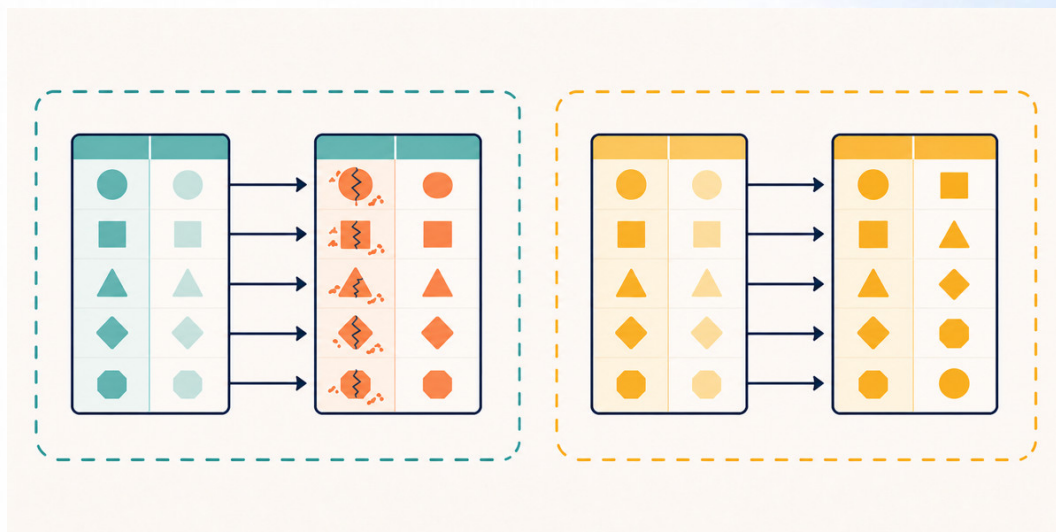


Figura 9. La clave natural describe. La sustituta solo identifica, y por eso no se rompe.

La regla práctica. Si hay un identificador oficial, estable y obligatorio, usalo. Si hay que dudar, poné una clave sustituta y guardá el dato natural como columna con unicidad exigida. Es la decisión más barata de tomar bien y la más cara de corregir después.

6.3 La clave compuesta

En el relevamiento, ¿qué identifica una fila?

id_hogar no alcanza: el hogar 1 tiene dos filas. fecha_visita tampoco: el 2025-03-16 hay dos visitas, a los hogares 4 y 5. Ninguna columna sola identifica una fila.

El par sí. id_hogar **más** fecha_visita identifica una única fila de las 45. Eso es una **clave compuesta**: una clave de más de una columna, donde ninguna de las dos alcanza sola.

id_hogar	fecha_visita	¿identifica?
id_hogar solo		no: el hogar 1 aparece dos veces
	fecha_visita sola	no: el 16 de marzo hay dos visitas
id_hogar	+ fecha_visita	sí, las 45 filas quedan distinguidas

Esta clave compuesta es el hallazgo central de la clase 1, y por dos razones.

La primera: **dice qué representa una fila**. Si la clave es el par hogar-fecha, entonces una fila no es un hogar: es una **visita**. El archivo se llama "relevamiento de hogares" y en realidad es un registro de visitas. La clave lo delata.

La segunda: es el origen del problema que resuelve la clase 3. Con una clave compuesta, algunas columnas dependen de las dos partes y otras de una sola. *personas* depende del par: el hogar 1 tenía tres personas en marzo y cuatro en junio. *localidad* depende solo de *id_hogar*: el hogar 1 está en Ramallo, y eso no cambia con la fecha. Esa asimetría tiene nombre y tiene cura, y son la segunda forma normal.

●	▲	●	■	◆
●	■	◆	●	▼
■	■	▼	◆	●
■	◆	●	▲	◆
◆	◆	■	▼	●
◆	▲	◆	●	■

Figura 10. Ninguna de las dos alcanza sola. El par identifica, y al identificar revela qué es una fila.

6.4 La clave foránea

Una **clave foránea** es una columna que guarda la clave primaria de otra tabla. Es el mecanismo con el que dos tablas se refieren.

En el diseño de la sección 3.4, *hogar.localidad_id* es clave foránea: sus valores son claves primarias de *localidad*. El hogar 1 dice *localidad_id* = 1, y el 1 de *localidad* es Ramallo.

Lo que hace útil a la clave foránea no es guardar el número. Es la regla que trae puesta: **todo valor de la clave foránea tiene que existir en la tabla a la que apunta**. Si alguien intenta cargar un hogar con *localidad_id* = 9, y no hay ninguna localidad 9, la base lo rechaza.

Esa regla se llama **integridad referencial**, y es el tema de la clase 2. Acá alcanza con retener tres cosas:

- La clave foránea no inventa datos: **copia una clave** que ya existe.
- La regla no es una recomendación: **la base la hace cumplir** en cada carga.
- Un archivo *.csv* no puede tener claves foráneas, porque no hay nadie que las controle.

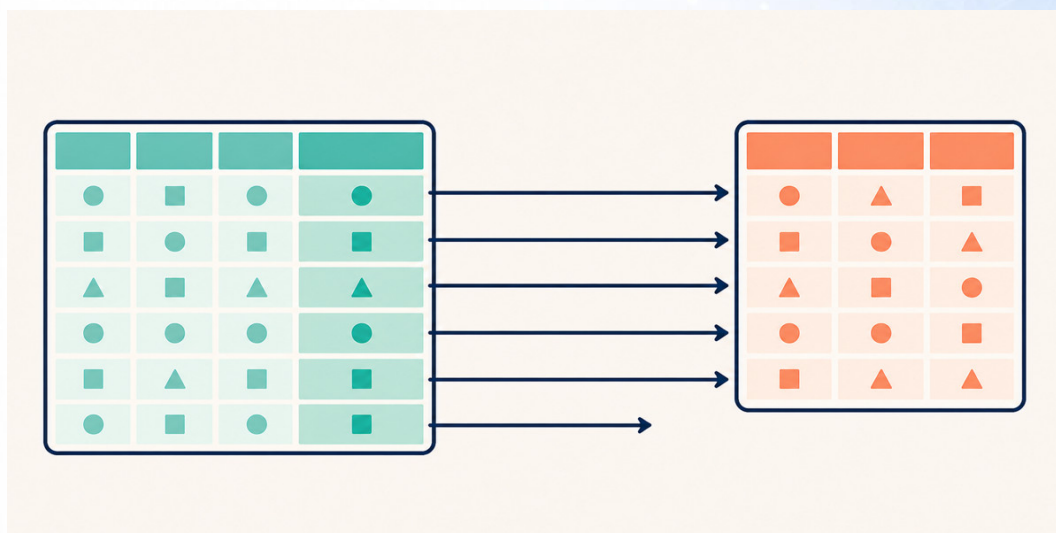


Figura 11. La flecha tiene que llegar a algún lado. La referencia que apunta al vacío no entra.

6.5 Cómo se reconoce una clave en una tabla que ya existe

En la práctica casi nunca diseñás desde cero: te llega una tabla y tenés que averiguar cuál es su clave. Cuatro pasos, en orden:

1. **Contá filas y contá valores distintos de cada columna.** Si una columna tiene tantos valores distintos como filas hay, es candidata. En el relevamiento ninguna lo es: `id_hogar` tiene 37 valores para 45 filas.
2. **Probá combinaciones de dos.** Ahí aparece el par `id_hogar` más `fecha_visita`, con 45 valores distintos para 45 filas. Y aparecen otros trece pares que también distinguen las 45 filas, porque `ingreso` y `fecha_visita` tienen casi un valor por fila. El paso 4 los descarta.
3. **Comprubá que sea mínima.** Si el par ya identifica, agregarle una tercera columna no mejora nada: la clave es el par y no el trío.
4. **Preguntá si la regla se sostiene en el futuro.** Esto no se contesta con datos. ¿Puede haber dos visitas al mismo hogar el mismo día? Si el operativo lo permite, la clave del paso 2 se rompe, y hace falta una clave sustituta `id_visita`. **Los datos que tenés muestran lo que pasó, no lo que puede pasar.**

El cuarto paso es el que más se saltea y el más importante. Una clave que funciona sobre las 45 filas de hoy y se rompe con la fila 46 no es una clave: es una coincidencia.

Y acá cierra el caso del T8001. Ese taller encontró dos filas que decían ser el hogar 41, con contenido distinto, y las descartó porque no había forma de desempatar. Con una clave primaria declarada sobre `id_hogar`, **la segunda fila nunca habría entrado**. El error se habría visto en la carga, cuando alguien todavía tenía el formulario en la mano y podía corregirlo. El taller no habría perdido un hogar.

Ese es el resumen de la clase en una frase: **una clave declarada convierte un problema de análisis en un problema de carga**, y los problemas de carga se resuelven.

7. QUÉ HACE UN DBMS

7.1 Las cinco funciones

Un **DBMS**, de *Database Management System*, o sistema de gestión de bases de datos, es el programa que administra una base de datos. No es la base: es lo que está entre la base y vos. PostgreSQL, MySQL, Oracle, SQL Server y SQLite son DBMS.

Nadie toca los archivos de una base de datos a mano. Se le pide al DBMS, y el DBMS hace cinco cosas:

Función	Qué resuelve
Consulta	Pedir un subconjunto de los datos sin abrir todo, describiendo qué se quiere
Integridad	Hacer cumplir las reglas declaradas, en cada carga y sin excepción
Concurrencia	Ordenar los accesos cuando varias personas leen y escriben a la vez
Seguridad	Decidir quién puede ver y cambiar qué
Recuperación	Dejar la base consistente aunque una operación se corte por la mitad

Ninguna de las cinco la puede dar un archivo, y todas son la razón por la que las organizaciones guardan su dato acá y no en una carpeta compartida.

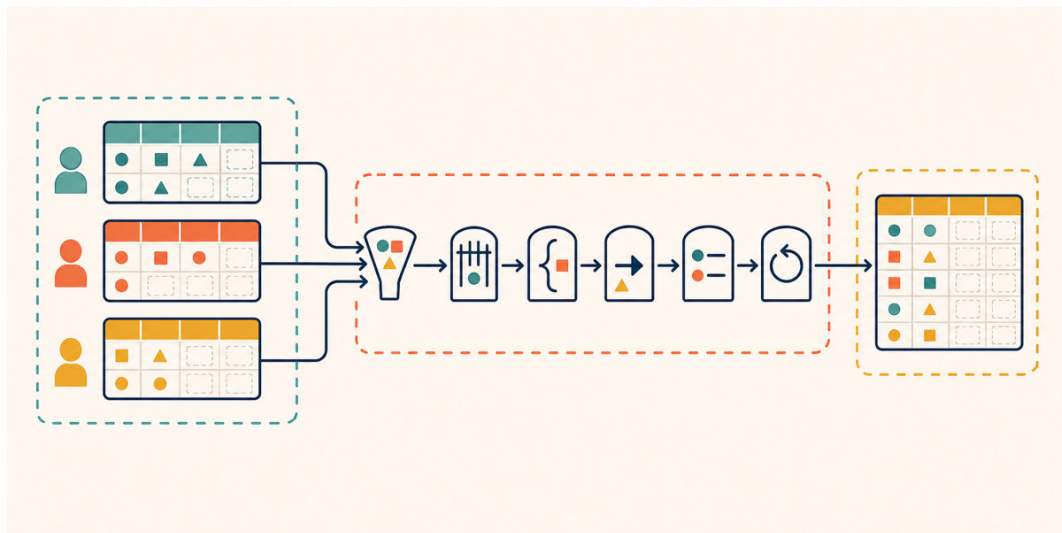


Figura 12. Nadie escribe en la base: todos le piden al DBMS, y el DBMS revisa cinco cosas.

7.2 Consulta e integridad

Consulta es la función más visible. A una base de datos no se le pide un archivo: se le describe un resultado. "Los hogares de Ramallo con más de cuatro personas, ordenados por ingreso". El DBMS decide cómo buscarlo y devuelve solo eso.

La diferencia con un archivo es de escala y de forma. Un .csv de dos millones de filas hay que leerlo entero para contestar cualquier pregunta. Una base de datos contesta esa misma pregunta sin recorrer todo, y le contesta a diez personas a la vez.

Integridad es hacer cumplir las reglas. Es la función que más aparece en este taller, porque es la que cambia el trabajo del analista. Las reglas son las de la sección 5.3 y la sección 6: el dominio de cada columna, la unicidad de la clave primaria y la validez de cada clave foránea. Están declaradas una vez, y el DBMS las revisa en cada carga.

Lo importante no es que las revise: es **cuándo**. Un archivo acepta el dato malo y el problema aparece meses después, en un promedio raro. Una base de datos lo rechaza en el momento, cuando todavía se puede arreglar.

7.3 Concurrencia, seguridad y recuperación

Las tres funciones que un archivo compartido no puede dar. Un párrafo cada una.

Concurrencia. Dos personas abren la misma planilla y cargan una visita cada una. La primera guarda, la segunda guarda encima, y la carga de la primera desaparece sin que nadie se entere. Eso se llama **actualización perdida**, y en una carpeta compartida pasa todos los días. Un DBMS ordena los turnos: mientras uno escribe una fila, el otro espera. El resultado es que las dos cargas existen. La palabra técnica para eso es **transacción**: un conjunto de operaciones que ocurre entero o no ocurre, y que para el resto del mundo pasa de una sola vez.

Seguridad. El permiso de un archivo es todo o nada: quien puede abrirlo ve todas las columnas y puede cambiar todas las filas. Un DBMS separa por usuario, por tabla y por columna. Al encuestador se le puede dar permiso para cargar visitas y no para ver ingresos; al área de análisis, permiso para leer todo y para cambiar nada. Además queda registro de quién hizo cada cosa, y esa es la diferencia entre un dato que se puede auditar y uno que no.

Recuperación. Se corta la luz en medio de una carga que toca tres tablas. En archivos, quedan dos tablas actualizadas y una no, y nadie sabe cuáles. Un DBMS lleva registro de lo que estaba haciendo, y al volver **deshace lo que quedó a medias**. La base queda como estaba antes de empezar, que es un estado consistente. Esa es la misma idea de la transacción: entero, o nada.

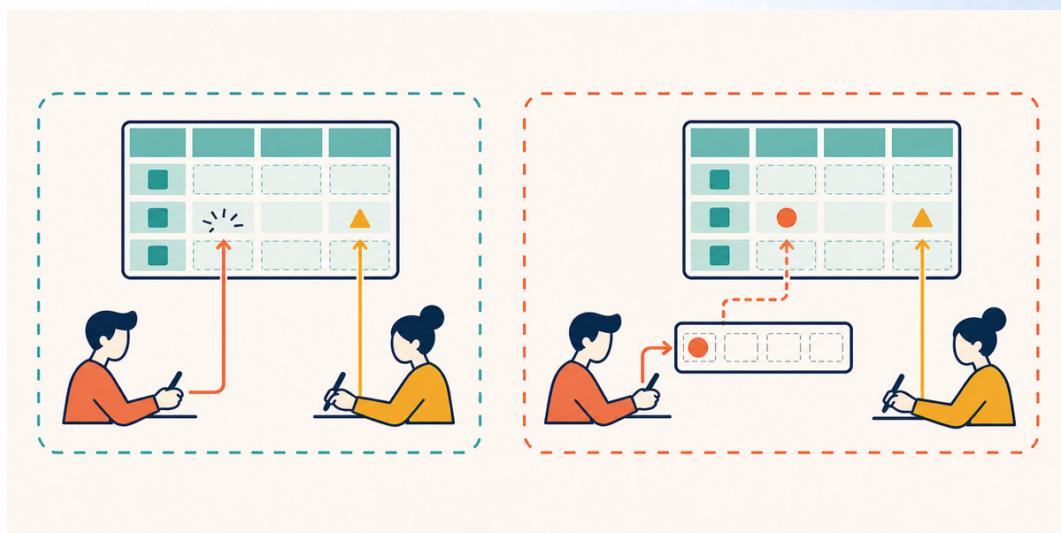


Figura 13. Sin turnos, la última escritura gana y la otra desaparece en silencio.

7.4 Relacional y no relacional, y por qué el taller mira el relacional

Los DBMS se agrupan en dos familias grandes.

Los **relacionales** guardan tablas con estructura declarada y filas que cumplen reglas. Son PostgreSQL, MySQL, Oracle, SQL Server y SQLite. Todo lo que dice este taller es de esta familia, y es la que se usa en la enorme mayoría de los sistemas de información de gestión.

Los **no relacionales**, o NoSQL, guardan otras formas: documentos, pares de clave y valor, grafos. Sirven para datos que no encajan en una tabla o para escalas muy grandes, y suelen relajar a propósito algunas de las reglas que un relacional exige.

Este taller mira el relacional por tres razones: es el que domina esos sistemas, es donde el vocabulario de claves y dependencias tiene sentido pleno, y es el que hace falta entender antes de entender por qué otro modelo decide apartarse de él. Las familias no relacionales y las bases analíticas son contenido de las asignaturas que siguen.

8. METADATOS Y ESQUEMA

8.1 Lo que la base sabe de sí misma

El T8001 ya trabajó con metadatos: el dato sobre el dato. Quién publicó un archivo, cuándo, con qué licencia, qué mide cada columna.

Lo nuevo acá no es el concepto: es **dónde viven**. En el T8001 los metadatos eran un documento al lado del archivo, escrito por una persona, que se podía perder, quedar viejo o contradecir al archivo sin que nada avisara. En una base de datos, los metadatos son parte de la base. Están guardados adentro, el DBMS los usa para trabajar, y no pueden contradecir a los datos porque son la regla que los datos cumplen.

Una base de datos sabe, sin que nadie se lo pregunte, qué tablas tiene, qué columnas tiene cada tabla, qué tipo y qué dominio tiene cada columna, cuál es la clave primaria y a dónde apunta cada clave foránea. Todo eso se puede consultar igual que se consultan los datos.

8.2 Del diccionario de variables al esquema

El **esquema** de una base de datos es la descripción completa de su estructura: las tablas, sus columnas, los tipos, los dominios, las claves y las referencias entre tablas.

Ahí hay un puente directo con el T8001, y también una palabra que cambió de tamaño.

El T8001 usó "esquema" en un sentido angosto, hablando de formatos de archivo: la ficha de tipos que un Parquet guarda junto a los datos y que un CSV no tiene. Era la mitad de la idea. El esquema de una base de datos es la versión completa: no solo qué tipo tiene cada columna, sino qué valores admite, cuál es la clave y qué otras tablas dependen de ella.

Y el diccionario de variables del T8001 es el borrador de ese esquema. Compará:

Columna del diccionario del T8001	Qué es en el esquema
variable	el nombre de la columna
descripcion	queda como comentario: la base no la hace cumplir
tipo y escala	orientan la elección del tipo, pero la escala no se declara
unidad	queda como comentario
valores_admitidos	el dominio , y la base lo hace cumplir
codigo_faltante	se reemplaza por el nulo: ya no hace falta un código
origen	queda como documentación

La lectura importante es la de la columna derecha. Parte del diccionario se convierte en **regla que se cumple sola**, y parte queda como documentación que sigue dependiendo de que alguien la escriba y la mantenga. El diccionario no desaparece: se parte en dos, y la mitad más frágil deja de ser frágil.

Un último par, que se confunde seguido. El **esquema** es la estructura: las tablas y sus reglas, y cambia poco. La **instancia** es el contenido en un momento dado: las filas que hay ahora, y

cambia todo el tiempo. Cuando alguien dice "la base de datos del relevamiento", casi siempre habla del esquema.

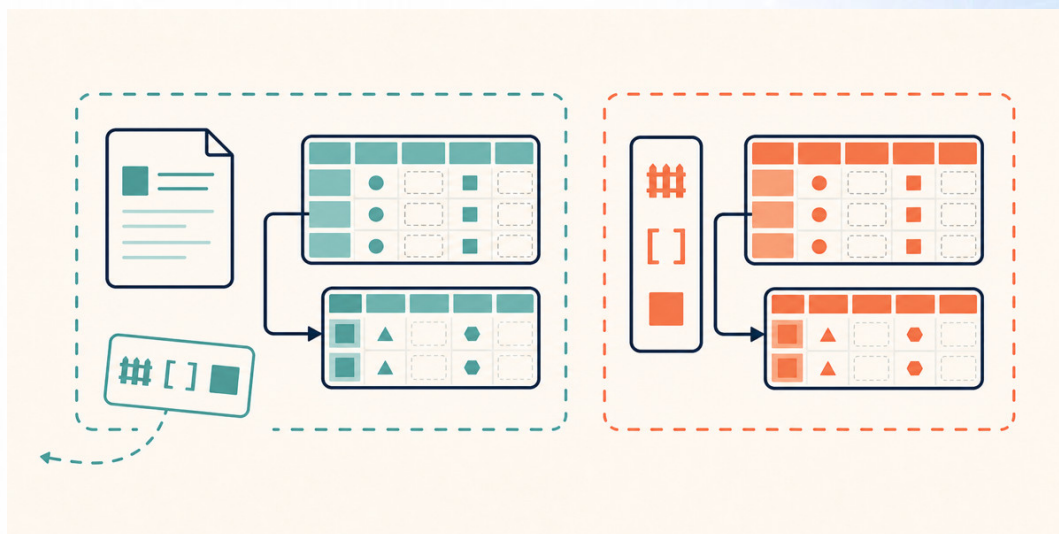


Figura 14. El diccionario al lado del archivo se puede perder. El esquema adentro de la base, no.

8.3 Lista de verificación

Antes de dar por buena la estructura de una tabla que te llega, cinco preguntas:

1. **¿Qué representa una fila?** No lo que dice el nombre del archivo. Lo que dice su clave.
2. **¿Cuál es la clave?** Probá columna por columna, después de a pares, y preguntá si la regla se sostiene con datos que todavía no llegaron.
3. **¿Qué hecho está escrito más de una vez?** Cada repetición es una anomalía de modificación esperando.
4. **¿Cuál es el dominio de cada columna, y quién lo hace cumplir?** Si la respuesta es "nadie", asumí que ya entró un valor imposible.
5. **¿Qué se pierde si borro una fila?** Si se va más de lo que querías borrar, la tabla mezcla cosas que van separadas.

9. LOS EJEMPLOS EN GOOGLE COLAB

El notebook de esta clase muestra en datos concretos todo lo anterior, en seis bloques que siguen el orden del apunte:

1. **La tabla de este taller.** Las 45 filas, las 12 columnas y los 37 hogares de la sección 1.3.
2. **La redundancia, contada.** Cuántas veces está escrito cada hecho, y cuántas copias de más son.
3. **Las tres anomalías.** Modificación, inserción y borrado, cada una sobre el relevamiento, y qué pasa cuando un cambio llega a dieciséis filas de diecisiete.
4. **El dominio que nadie controla.** Un personas = -3 y un provincia = "Ramallo" entran sin una queja.
5. **La búsqueda de la clave.** Los cuatro pasos de la sección 6.5, hasta encontrar el par id_hogar más fecha_visita.
6. **Las dos tablas, y el cruce.** Separar localidad del relevamiento, y volver a juntarlas con un merge para comprobar que no se perdió nada.

Recordá que no hay nada que completar. Ejecutá de arriba hacia abajo y leé cada salida: [01 bases y claves.ipynb](#).